

Lab Report – Automated Product Listing Generator (ChatGPT Vision API)

Author: Elif Yurdakul · **Model:** gpt-4o-mini · **Dataset:** ashraq/fashion-product-images-small (Hugging Face)

1. How the integration works

`product_listing_generator.py` turns a product image plus basic metadata into a ready-to-publish listing in seven steps:

1. **Authentication:** the API key is read from an environment variable (`.env` via `python-dotenv`). The key is never in the code, `.env` is in `.gitignore`, and only a masked version (`sk-sv...s4AA`) is ever printed.
2. **Dataset:** the first N products are loaded from Hugging Face. Images are saved as JPEGs, and columns are mapped to `name` / `price` / `category` / `image_path`. The dataset has no prices, so a realistic, reproducible price is assigned per article type. If Hugging Face is unreachable, the script falls back to local images.
3. **Encoding:** each image is validated (exists, supported format, not empty) and converted to base64.
4. **Prompt:** a copywriter role, the product data (including colour, gender, season and usage), the format rules and a JSON template.
5. **API call:** text and image are sent in one message (`detail: "low"`, `response_format: json_object`). Transient errors (rate limit, timeout, connection, 5xx) are retried with exponential backoff (2 s, 4 s, 8 s). Permanent errors (invalid key, bad request) are not retried.
6. **Batch processing:** a failing product is recorded as `"status": "error"` without stopping the batch, and results are saved to `generated_listings.json` after every product.
7. **Quality and cost (bonus 3 + 4):** each listing is checked against the brief (title ≤ 60 characters, 150–200 words, 5–7 features, 10–15 keywords). Short descriptions are expanded automatically with a cheap text-only follow-up request, and tokens and cost are tracked per listing.

Results: 5/5 listings generated, 0 failures, 0 flagged, ~3,500 tokens and ~\$0.00067 per listing (≈ **\$0.67 per 1,000 listings**). The error-handling demo shows 5 handled cases: missing file, unsupported format, markdown-wrapped JSON, invalid key (401) and missing key.

2. Challenges

- **The dataset did not match the template.** The Hugging Face rows have no `price` or `image_path`, so the template code would have failed in Step 3. I added image saving, column mapping and price assignment.
- **A large download for a small sample.** Loading 5 rows downloads the full parquet shards (~135 MB each) once. They are cached afterwards.
- **Environment issues.** The VS Code terminal crashed ("Pty Host"), and Conda (base) and `.venv` were active at the same time. Fixed by reloading the window and running `conda deactivate`.
- **Images are very small** (~60×80 px, ~2.4 KB as base64), which limits visible detail. Still, the image tokens make up most of the ~3,400 prompt tokens per call.

3. Quality of the generated listings

Version	Prompt change	Description length	Flagged	Invented claims
v1	Lab template	105–124 words	5/5	"soft, breathable fabric" on the shirt
v2	+ "do not claim fabric/material unless given"	115–124 words	5/5	reduced; still "Water-resistant" (watch), "breathable" (T-shirt)
v3	+ "MINIMUM 150 words, ~175 words in 3 short paragraphs"	160–179 words	0/5	"breathability / lightweight fabric" remains on the T-shirt

Titles (43–52 characters), features and keywords met the brief in every run. The model correctly used visual details that were not in the metadata, such as the check pattern, collar and sleeve length.

Key findings:

1. A range like "150–200 words" was treated as an upper limit. An explicit minimum, a target number and a paragraph structure fixed the length completely.
2. Instructions *reduce* hallucination but do not eliminate it. Even the word "breathable", which the prompt names explicitly as forbidden, reappeared. Factual claims therefore still need human review before publishing.

4. Potential improvements

- Add a **fact-check pass** that flags material or performance claims (cotton, waterproof, breathable...) not present in the metadata, and route those listings to human review.
- Use **Structured Outputs** (a strict JSON schema) instead of `json_object`.
- Test **higher-resolution images** or `detail: "high"`, and compare **gpt-4o vs. gpt-4o-mini** on quality per dollar.
- **Process in parallel** (async) for large catalogues, and export to **CSV with SKUs** for the e-commerce platform.
- **Style variants** (formal/casual, by target audience) for A/B testing.